

\$\$\$ sql.banco

DROP TABLE IF EXISTS Seguradoras;

DROP TABLE IF EXISTS Pecas;

```
CREATE TABLE Seguradoras (  
    Nome VARCHAR(50) NOT NULL PRIMARY KEY,  
    Cidade VARCHAR(50),  
    CoberturaPercentual DECIMAL(5,2) NOT NULL  
);
```

```
CREATE TABLE Pecas (  
    Codigo INT NOT NULL PRIMARY KEY,  
    Nome VARCHAR(50) NOT NULL,  
    Categoria VARCHAR(50) NOT NULL,  
    Preco DECIMAL(10,2) NOT NULL,  
    Tipo VARCHAR(50) NOT NULL,  
    Cor VARCHAR(15) NOT NULL,  
    MaoDeObra BOOLEAN NOT NULL  
);
```

\$\$\$ src.controles.ControladorCadastroPecas

```
seguradoras = {}
```

```
def get_seguradoras():
```

```
    return seguradoras
```

```
def inserir_seguradora(seguradora):
```

```
    nome_seguradora = seguradora.nome
```

```
    if nome_seguradora not in seguradoras.keys():
```

```
        seguradoras[nome_seguradora] = seguradora
```

```
    return True
```

```
    else:
```

```
        print('Seguradora ' + nome_seguradora + ' tem cadastro')
```

```
    return False
```

```
def set_seguradoras(seguradoras1):
```

```
    global seguradoras
```

```
    seguradoras = seguradoras1
```

```
class Seguradora:
```

```
    def __init__(self, nome, cidade, cobertura_percentual):
```

```
        self.nome = nome
```

```
        self.cidade = cidade
```

```
        self.cobertura_percentual = cobertura_percentual
```

```
    def __str__(self):
```

```
        formato = '{ } {:<18} { } {:<14} { } {:<3} { }'
```

```
        seguradora_formatada = formato.format('|', self.nome, '|', self.cidade, '|',  
str(self.cobertura_percentual), '|')
```

```
        return seguradora_formatada
```

\$\$\$ src.controles.ControladorCadastroSeguradoras

```
package controles;

import entidades.Seguradora;
import interfaces.JanelaCadastroSeguradoras;
import java.awt.Frame;

public class ControladorCadastroSeguradoras {

    public ControladorCadastroSeguradoras() {
        this(null);
    }

    public ControladorCadastroSeguradoras(Frame owner) {
        JanelaCadastroSeguradoras janela = new JanelaCadastroSeguradoras(this, owner);
        janela.setVisible(true);
        janelaToFront();
        janela.requestFocus();
    }

    public String inserirSeguradora(Seguradora seguradora) {
        Seguradora seguradora_buscada = Seguradora.buscarSeguradora(seguradora.getNome());
        if (seguradora_buscada == null) return Seguradora.inserirSeguradora(seguradora);
        else return "Nome de Seguradora já cadastrado";
    }

    public String alterarSeguradora(Seguradora seguradora) {
        Seguradora seguradora_buscada = Seguradora.buscarSeguradora(seguradora.getNome());
        if (seguradora_buscada != null) return Seguradora.alterarSeguradora(seguradora);
        else return "Nome de Seguradora não cadastrado";
    }

    public String removerSeguradora(String nome) {
        Seguradora seguradora_buscada = Seguradora.buscarSeguradora(nome);
        if (seguradora_buscada != null) return Seguradora.removerSeguradora(nome);
        else return "Nome de Seguradora não cadastrado";
    }
}
```

}

}

\$\$\$ src.entidades.Pecas

```
package entidades;

import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.util.ArrayList;
import persistência.BD;

public class Pecas {

    private int codigo;
    private String nome;
    private String categoria;
    private double preco;
    private String tipo;
    private String cor;
    private boolean mao_de_obra;

    public int getCodigo() {
        return codigo;
    }

    public String getNome() {
        return nome;
    }

    public String getCategoria() {
        return categoria;
    }

    public double getPreco() {
        return preco;
    }

    public String getTipo() {
```

```
    return tipo;
}
```

```
public String getCor() {
    return cor;
}
```

```
public boolean getMaoDeObra() {
    return mao_de_obra;
}
```

```
public void setCodigo(int codigo) {
    this.codigo = codigo;
}
```

```
public void setNome(String nome) {
    this.nome = nome;
}
```

```
public void setCategoria(String categoria) {
    this.categoria = categoria;
}
```

```
public void setPreco(double preco) {
    this.preco = preco;
}
```

```
public void setTipo(String tipo) {
    this.tipo = tipo;
}
```

```
public void setCor(String cor) {
    this.cor = cor;
}
```

```
public void setMaoDeObra(boolean mao_de_obra) {
    this.mao_de_obra = mao_de_obra;
}
```

```
}
```

```
public Pecas(int codigo, String nome, String categoria,  
             double preco, String tipo, String cor,  
             boolean mao_de_obra) {
```

```
  
    this.codigo = codigo;  
    this.nome = nome;  
    this.categoria = categoria;  
    this.preco = preco;  
    this.tipo = tipo;  
    this.cor = cor;  
    this.mao_de_obra = mao_de_obra;  
}
```

```
public String toString() {  
    return codigo + " - " + nome + " [" + categoria + "];  
}
```

```
public static Pecas[] getVisoes() {
```

```
  
    String sql = "SELECT Codigo, Nome FROM Pecas";
```

```
  
    ResultSet lista_resultados = null;  
    ArrayList<Pecas> visoes = new ArrayList();
```

```
    try {
```

```
  
        PreparedStatement comando =  
            BD.conexao.prepareStatement(sql);
```

```
  
        lista_resultados = comando.executeQuery();
```

```
  
        while (lista_resultados.next()) {
```

```
  
            int codigo =  
                lista_resultados.getInt("Codigo");
```

```

        String nome =
            lista_resultados.getString("Nome");

        visoes.add(
            new Pecas(codigo, nome, null,
                0, null, null, false)
        );
    }

    lista_resultados.close();
    comando.close();

} catch (SQLException excecao_sql) {
    excecao_sql.printStackTrace();
}

return visoes.toArray(new Pecas[visoes.size()]);
}

public Pecas getVisao() {
    return new Pecas(
        codigo,
        nome,
        categoria,
        preco,
        tipo,
        cor,
        mao_de_obra
    );
}

public static Pecas buscarPecas(String nome) {

    String sql =
        "SELECT * FROM Pecas WHERE Nome = ?";

```

```
ResultSet lista_resultados = null;
Pecas pecas = null;

try {

    PreparedStatement comando =
        BD.conexao.prepareStatement(sql);

    comando.setString(1, nome);

    lista_resultados = comando.executeQuery();

    while (lista_resultados.next()) {

        pecas = new Pecas(
            lista_resultados.getInt("Codigo"),
            lista_resultados.getString("Nome"),
            lista_resultados.getString("Categoria"),
            lista_resultados.getDouble("Preco"),
            lista_resultados.getString("Tipo"),
            lista_resultados.getString("Cor"),
            lista_resultados.getBoolean("MaoDeObra")
        );
    }

    lista_resultados.close();
    comando.close();

} catch (SQLException excecao_sql) {

    excecao_sql.printStackTrace();
    pecas = null;
}

return pecas;
}
```

```

public static String inserirPecas(Pecas pecas) {

    String sql =
        "INSERT INTO Pecas " +
        "(Codigo, Nome, Categoria, Preco, Tipo, Cor, MaoDeObra) " +
        "VALUES (?, ?, ?, ?, ?, ?, ?)";

    try {

        PreparedStatement comando =
            BD.conexao.prepareStatement(sql);

        comando.setInt(1, pecas.getCodigo());
        comando.setString(2, pecas.getNome());
        comando.setString(3, pecas.getCategoria());
        comando.setDouble(4, pecas.getPreco());
        comando.setString(5, pecas.getTipo());
        comando.setString(6, pecas.getCor());
        comando.setBoolean(7, pecas.getMaoDeObra());

        comando.executeUpdate();
        comando.close();

        return null;

    } catch (SQLException excecao_sql) {

        excecao_sql.printStackTrace();

        return "Erro na Inserção da Peça no BD";
    }
}

```

```

public static String alterarPecas(Pecas pecas) {

    String sql =
        "UPDATE Pecas SET " +

```

```

        "Codigo = ?, " +
        "Categoria = ?, " +
        "Preco = ?, " +
        "Tipo = ?, " +
        "Cor = ?, " +
        "MaoDeObra = ? " +
        "WHERE Nome = ?";

try {

    PreparedStatement comando =
        BD.conexao.prepareStatement(sql);

    comando.setInt(1, pecas.getCodigo());
    comando.setString(2, pecas.getCategoria());
    comando.setDouble(3, pecas.getPreco());
    comando.setString(4, pecas.getTipo());
    comando.setString(5, pecas.getCor());
    comando.setBoolean(6, pecas.getMaoDeObra());
    comando.setString(7, pecas.getNome());

    comando.executeUpdate();
    comando.close();

    return null;

} catch (SQLException excecao_sql) {

    excecao_sql.printStackTrace();

    return "Erro na Alteração da Peça no BD";
}

}

public static String removerPecas(String nome) {

    String sql =

```

```
"DELETE FROM Pecas WHERE Nome = ?";
```

```
try {
```

```
    PreparedStatement comando =
```

```
        BD.conexao.prepareStatement(sql);
```

```
    comando.setString(1, nome);
```

```
    comando.executeUpdate();
```

```
    comando.close();
```

```
    return null;
```

```
} catch (SQLException excecao_sql) {
```

```
    excecao_sql.printStackTrace();
```

```
    return "Erro na Remoção da Peça no BD";
```

```
}
```

```
}
```

```
}
```

\$\$\$ src.entidades.Seguradora

```
package entidades;
```

```
import java.sql.PreparedStatement;
```

```
import java.sql.ResultSet;
```

```
import java.sql.SQLException;
```

```
import persistência.BD;
```

```
public class Seguradora {
```

```
    public String getNome() { return nome; }
```

```
    public String getCidade() { return cidade; }
```

```
    public double getCoberturaPercentual() { return cobertura_percentual; }
```

```
    public void setCidade(String cidade) { this.cidade = cidade; }
```

```
    public void setCoberturaPercentual(double cobertura_percentual) { this.cobertura_percentual =  
cobertura_percentual; }
```

```
    private String nome, cidade;
```

```
    private double cobertura_percentual;
```

```
    public Seguradora(String nome, String cidade, double cobertura_percentual) {
```

```
        this.nome = nome;
```

```
        this.cidade = cidade;
```

```
        this.cobertura_percentual = cobertura_percentual;
```

```
    }
```

```
    public String toString() {
```

```
        return nome + " [" + cidade + "];
```

```
    }
```

```
    public Seguradora getVisao() {
```

```
        return new Seguradora(nome, cidade);
```

```
    }
```

```
    public Seguradora(String nome, String cidade) {
```

```
    this.nome = nome;
    this.cidade = cidade;
}
```

```
public static Seguradora[] getVisoes() {
    String sql = "SELECT Nome, Cidade FROM Seguradoras";
    ResultSet lista_resultados = null;
    java.util.ArrayList<Seguradora> visoes = new java.util.ArrayList();

    try {
        PreparedStatement comando = BD.conexao.prepareStatement(sql);
        lista_resultados = comando.executeQuery();

        while (lista_resultados.next()) {
            String nome = lista_resultados.getString("Nome");
            String cidade = lista_resultados.getString("Cidade");
            visoes.add(new Seguradora(nome, cidade));
        }

        lista_resultados.close();
        comando.close();

    } catch (SQLException excecao_sql) {
        excecao_sql.printStackTrace();
    }

    return visoes.toArray(new Seguradora[visoes.size()]);
}
```

```
public static Seguradora buscarSeguradora(String nome) {
    String sql = "SELECT * FROM Seguradoras WHERE Nome = ?";
    ResultSet lista_resultados = null;
    Seguradora seguradora = null;

    try {
        PreparedStatement comando = BD.conexao.prepareStatement(sql);
        comando.setString(1, nome);
```

```
lista_resultados = comando.executeQuery();
```

```
while (lista_resultados.next()) {  
    seguradora = new Seguradora(  
        nome,  
        lista_resultados.getString("Cidade"),  
        lista_resultados.getDouble("CoberturaPercentual")  
    );  
}
```

```
lista_resultados.close();  
comando.close();
```

```
} catch (SQLException excecao_sql) {  
    excecao_sql.printStackTrace();  
    seguradora = null;  
}
```

```
return seguradora;  
}
```

```
public static String inserirSeguradora(Seguradora seguradora) {
```

```
    String sql = "INSERT INTO Seguradoras (Nome, Cidade, CoberturaPercentual) VALUES  
(?,?,?)";
```

```
try {  
    PreparedStatement comando = BD.conexao.prepareStatement(sql);
```

```
    comando.setString(1, seguradora.getNome());  
    comando.setString(2, seguradora.getCidade());  
    comando.setDouble(3, seguradora.getCoberturaPercentual());
```

```
    comando.executeUpdate();  
    comando.close();
```

```
    return null;
```

```

    } catch (SQLException excecao_sql) {
        excecao_sql.printStackTrace();
        return "Erro na Inserção da Seguradora no BD";
    }
}

```

```

public static String alterarSeguradora(Seguradora seguradora) {
    String sql = "UPDATE Seguradoras SET Cidade = ?, CoberturaPercentual = ?"
        + " WHERE Nome = ?";

    try {
        PreparedStatement comando = BD.conexao.prepareStatement(sql);

        comando.setString(1, seguradora.getCidade());
        comando.setDouble(2, seguradora.getCoberturaPercentual());
        comando.setString(3, seguradora.getNome());

        comando.executeUpdate();
        comando.close();

        return null;

    } catch (SQLException excecao_sql) {
        excecao_sql.printStackTrace();
        return "Erro na Alteração da Seguradora no BD";
    }
}

```

```

public static String removerSeguradora(String nome) {
    String sql = "DELETE FROM Seguradoras WHERE Nome = ?";

    try {
        PreparedStatement comando = BD.conexao.prepareStatement(sql);

        comando.setString(1, nome);
        comando.executeUpdate();
        comando.close();
    }
}

```

```
        return null;

    } catch (SQLException excecao_sql) {
        excecao_sql.printStackTrace();
        return "Erro na Remoção da Seguradora no BD";
    }
}
}
```

\$\$\$ src.interfaces.JanelaCadastroPecas

```
package interfaces;

import javax.swing.JOptionPane;
import controles.ControladorCadastroPecas;
import entidades.Pecas;
import javax.swing.DefaultComboBoxModel;
import java.awt.Frame;

public class JanelaCadastroPecas extends javax.swing.JFrame {

    ControladorCadastroPecas controlador;
    Pecas[] pecas_cadastradas;

    public JanelaCadastroPecas(ControladorCadastroPecas controlador) {
        this(controlador, null);
    }

    public JanelaCadastroPecas(
        ControladorCadastroPecas controlador,
        Frame owner
    ) {
        this.controlador = controlador;
        pecas_cadastradas = Pecas.getVisoes();
        initComponents();
        setSize(new java.awt.Dimension(720, 560));
        configurarJanelaDependente(owner);
        limparCampos(null);
    }

    private void configurarJanelaDependente(Frame owner) {
        setAutoRequestFocus(true);
        setAlwaysOnTop(true);

        if (owner == null) {
            setLocationByPlatform(true);
        }
    }
}
```

```

        return;
    }

    posicionarRelativaAoOwner(owner);
}

private void posicionarRelativaAoOwner(Frame owner) {
    int x = owner.getX() + (owner.getWidth() - getWidth()) / 2;
    int y = owner.getY() + (owner.getHeight() - getHeight()) / 2;
    setLocation(x, y);
}

private void informarErro(String mensagem) {
    JOptionPane.showMessageDialog(
        this,
        mensagem,
        "Erro",
        JOptionPane.ERROR_MESSAGE
    );
}

private Pecas obterPecasInformada() {
    String codigoStr = codigoTextField.getText();
    if (codigoStr.isEmpty()) return null;

    int codigo;
    try {
        codigo = Integer.parseInt(codigoStr);
    } catch (NumberFormatException e) {
        return null;
    }

    String nome = nomeTextField.getText();
    if (nome.isEmpty()) return null;

    String categoria = categoriaTextField.getText();
    if (categoria.isEmpty()) return null;
}

```

```
String precoStr = precoTextField.getText();  
if (precoStr.isEmpty()) return null;
```

```
double preco;  
try {  
    preco = Double.parseDouble(precoStr.replace(",", "."));  
} catch (NumberFormatException e) {  
    return null;  
}
```

```
String tipo = tipoTextField.getText();  
if (tipo.isEmpty()) return null;
```

```
String cor = corTextField.getText();  
if (cor.isEmpty()) return null;
```

```
String mao_de_obra_str = maoDeObraTextField.getText();  
if (mao_de_obra_str.isEmpty()) return null;
```

```
boolean mao_de_obra;
```

```
if (mao_de_obra_str.equalsIgnoreCase("S")  
    || mao_de_obra_str.equalsIgnoreCase("SIM")  
    || mao_de_obra_str.equalsIgnoreCase("TRUE")) {  
    mao_de_obra = true;  
} else if (mao_de_obra_str.equalsIgnoreCase("N")  
    || mao_de_obra_str.equalsIgnoreCase("NAO")  
    || mao_de_obra_str.equalsIgnoreCase("NÃO")  
    || mao_de_obra_str.equalsIgnoreCase("FALSE")) {  
    mao_de_obra = false;  
} else {  
    return null;  
}
```

```
return new Pecas(  
    codigo,
```

```
        nome,  
        categoria,  
        preco,  
        tipo,  
        cor,  
        mao_de_obra  
    );  
}
```

```
private Pecas getVisaoAlterada(String nome) {  
    for (Pecas visao : pecas_cadastradas) {  
        if (visao.getNome().equals(nome)) return visao;  
    }  
    return null;  
}
```

```
@SuppressWarnings("unchecked")
```

```
// <editor-fold defaultstate="collapsed" desc="Generated Code">//GEN-BEGIN: initComponents
```

```
private void initComponents() {  
    java.awt.GridBagConstraints gridBagConstraints;  
  
    comandosPanel = new javax.swing.JPanel();  
    inserirPecas = new javax.swing.JButton();  
    alterarPecas = new javax.swing.JButton();  
    consultarPecas = new javax.swing.JButton();  
    removerPecas = new javax.swing.JButton();  
    limparCampos = new javax.swing.JButton();  
  
    codigoLabel = new javax.swing.JLabel();  
    codigoTextField = new javax.swing.JTextField();  
  
    nomeLabel = new javax.swing.JLabel();  
    nomeTextField = new javax.swing.JTextField();  
  
    categoriaLabel = new javax.swing.JLabel();  
    categoriaTextField = new javax.swing.JTextField();  
}
```

```
precoLabel = new javax.swing.JLabel();
precoTextField = new javax.swing.JTextField();

tipoLabel = new javax.swing.JLabel();
tipoTextField = new javax.swing.JTextField();

corLabel = new javax.swing.JLabel();
corTextField = new javax.swing.JTextField();

maoDeObraLabel = new javax.swing.JLabel();
maoDeObraTextField = new javax.swing.JTextField();

pecas_cadastradasComboBox = new javax.swing.JComboBox();
pecasCadastradasLabel = new javax.swing.JLabel();

setDefaultCloseOperation(javax.swing.WindowConstants.DISPOSE_ON_CLOSE);
setTitle("Cadastrar Peças");
setMinimumSize(new java.awt.Dimension(640, 420));
setPreferredSize(new java.awt.Dimension(720, 560));

java.awt.GridBagLayout layout = new java.awt.GridBagLayout();
layout.rowHeights = new int[] {8};
getContentPane().setLayout(layout);

inserirPecas.setText("Inserir");
inserirPecas.addActionListener(new java.awt.event.ActionListener() {
    public void actionPerformed(java.awt.event.ActionEvent evt) {
        inserirPecas(evt);
    }
});
comandosPanel.add(inserirPecas);

alterarPecas.setText("Alterar");
alterarPecas.addActionListener(new java.awt.event.ActionListener() {
    public void actionPerformed(java.awt.event.ActionEvent evt) {
        alterarPecas(evt);
    }
})
```

```
});  
comandosPanel.add(alterarPecas);  
  
consultarPecas.setText("Consultar");  
consultarPecas.addActionListener(new java.awt.event.ActionListener() {  
    public void actionPerformed(java.awt.event.ActionEvent evt) {  
        consultarPecas(evt);  
    }  
});  
comandosPanel.add(consultarPecas);  
  
removerPecas.setText("Remover");  
removerPecas.addActionListener(new java.awt.event.ActionListener() {  
    public void actionPerformed(java.awt.event.ActionEvent evt) {  
        removerPecas(evt);  
    }  
});  
comandosPanel.add(removerPecas);  
  
limparCampos.setText("Limpar");  
limparCampos.addActionListener(new java.awt.event.ActionListener() {  
    public void actionPerformed(java.awt.event.ActionEvent evt) {  
        limparCampos(evt);  
    }  
});  
comandosPanel.add(limparCampos);  
  
gridBagConstraints = new java.awt.GridBagConstraints();  
gridBagConstraints.gridx = 1;  
gridBagConstraints.gridy = 9;  
gridBagConstraints.insets = new java.awt.Insets(5, 5, 5, 5);  
getContentPane().add(comandosPanel, gridBagConstraints);  
  
pecasCadastradasLabel.setText("Peças Cadastradas");  
gridBagConstraints = new java.awt.GridBagConstraints();  
gridBagConstraints.gridx = 0;  
gridBagConstraints.gridy = 0;
```

```
gridBagConstraints.anchor = java.awt.GridBagConstraints.EAST;
gridBagConstraints.insets = new java.awt.Insets(1, 5, 10, 5);
getContentPane().add(pecasCadastradasLabel, gridBagConstraints);
```

```
pecas_cadastradasComboBox.setModel(
    new DefaultComboBoxModel(pecas_cadastradas)
);
```

```
gridBagConstraints = new java.awt.GridBagConstraints();
gridBagConstraints.gridx = 1;
gridBagConstraints.gridy = 0;
gridBagConstraints.anchor = java.awt.GridBagConstraints.NORTHWEST;
gridBagConstraints.insets = new java.awt.Insets(1, 5, 10, 5);
getContentPane().add(pecas_cadastradasComboBox, gridBagConstraints);
```

```
codigoLabel.setText("Código");
gridBagConstraints = new java.awt.GridBagConstraints();
gridBagConstraints.gridx = 0;
gridBagConstraints.gridy = 1;
gridBagConstraints.anchor = java.awt.GridBagConstraints.EAST;
gridBagConstraints.insets = new java.awt.Insets(1, 5, 10, 5);
getContentPane().add(codigoLabel, gridBagConstraints);
```

```
codigoTextField.setColumns(8);
codigoTextField.setPreferredSize(new java.awt.Dimension(80, 20));
gridBagConstraints = new java.awt.GridBagConstraints();
gridBagConstraints.gridx = 1;
gridBagConstraints.gridy = 1;
gridBagConstraints.anchor = java.awt.GridBagConstraints.WEST;
gridBagConstraints.insets = new java.awt.Insets(1, 5, 10, 5);
getContentPane().add(codigoTextField, gridBagConstraints);
```

```
nomeLabel.setText("Nome");
gridBagConstraints = new java.awt.GridBagConstraints();
gridBagConstraints.gridx = 0;
gridBagConstraints.gridy = 2;
gridBagConstraints.anchor = java.awt.GridBagConstraints.EAST;
```

```
gridBagConstraints.insets = new java.awt.Insets(1, 5, 10, 5);  
getContentPane().add(nomeLabel, gridBagConstraints);
```

```
nomeTextField.setColumns(24);  
nomeTextField.setPreferredSize(new java.awt.Dimension(240, 20));  
gridBagConstraints = new java.awt.GridBagConstraints();  
gridBagConstraints.gridx = 1;  
gridBagConstraints.gridy = 2;  
gridBagConstraints.anchor = java.awt.GridBagConstraints.WEST;  
gridBagConstraints.insets = new java.awt.Insets(1, 5, 10, 5);  
getContentPane().add(nomeTextField, gridBagConstraints);
```

```
categoriaLabel.setText("Categoria");  
gridBagConstraints = new java.awt.GridBagConstraints();  
gridBagConstraints.gridx = 0;  
gridBagConstraints.gridy = 3;  
gridBagConstraints.anchor = java.awt.GridBagConstraints.EAST;  
gridBagConstraints.insets = new java.awt.Insets(1, 5, 10, 5);  
getContentPane().add(categoriaLabel, gridBagConstraints);
```

```
categoriaTextField.setColumns(18);  
categoriaTextField.setPreferredSize(new java.awt.Dimension(180, 20));  
gridBagConstraints = new java.awt.GridBagConstraints();  
gridBagConstraints.gridx = 1;  
gridBagConstraints.gridy = 3;  
gridBagConstraints.anchor = java.awt.GridBagConstraints.WEST;  
gridBagConstraints.insets = new java.awt.Insets(1, 5, 10, 5);  
getContentPane().add(categoriaTextField, gridBagConstraints);
```

```
precoLabel.setText("Preço");  
gridBagConstraints = new java.awt.GridBagConstraints();  
gridBagConstraints.gridx = 0;  
gridBagConstraints.gridy = 4;  
gridBagConstraints.anchor = java.awt.GridBagConstraints.EAST;  
gridBagConstraints.insets = new java.awt.Insets(1, 5, 10, 5);  
getContentPane().add(precoLabel, gridBagConstraints);
```

```
precoTextField.setColumns(10);
precoTextField.setPreferredSize(new java.awt.Dimension(100, 20));
gridBagConstraints = new java.awt.GridBagConstraints();
gridBagConstraints.gridx = 1;
gridBagConstraints.gridy = 4;
gridBagConstraints.anchor = java.awt.GridBagConstraints.WEST;
gridBagConstraints.insets = new java.awt.Insets(1, 5, 10, 5);
getContentPane().add(precoTextField, gridBagConstraints);
```

```
tipoLabel.setText("Tipo");
gridBagConstraints = new java.awt.GridBagConstraints();
gridBagConstraints.gridx = 0;
gridBagConstraints.gridy = 5;
gridBagConstraints.anchor = java.awt.GridBagConstraints.EAST;
gridBagConstraints.insets = new java.awt.Insets(1, 5, 10, 5);
getContentPane().add(tipoLabel, gridBagConstraints);
```

```
tipoTextField.setColumns(14);
tipoTextField.setPreferredSize(new java.awt.Dimension(140, 20));
gridBagConstraints = new java.awt.GridBagConstraints();
gridBagConstraints.gridx = 1;
gridBagConstraints.gridy = 5;
gridBagConstraints.anchor = java.awt.GridBagConstraints.WEST;
gridBagConstraints.insets = new java.awt.Insets(1, 5, 10, 5);
getContentPane().add(tipoTextField, gridBagConstraints);
```

```
corLabel.setText("Cor");
gridBagConstraints = new java.awt.GridBagConstraints();
gridBagConstraints.gridx = 0;
gridBagConstraints.gridy = 6;
gridBagConstraints.anchor = java.awt.GridBagConstraints.EAST;
gridBagConstraints.insets = new java.awt.Insets(1, 5, 10, 5);
getContentPane().add(corLabel, gridBagConstraints);
```

```
corTextField.setColumns(10);
corTextField.setPreferredSize(new java.awt.Dimension(100, 20));
gridBagConstraints = new java.awt.GridBagConstraints();
```

```
gridBagConstraints.gridx = 1;
gridBagConstraints.gridy = 6;
gridBagConstraints.anchor = java.awt.GridBagConstraints.WEST;
gridBagConstraints.insets = new java.awt.Insets(1, 5, 10, 5);
getContentPane().add(corTextField, gridBagConstraints);
```

```
maoDeObraLabel.setText("Mão de Obra");
gridBagConstraints = new java.awt.GridBagConstraints();
gridBagConstraints.gridx = 0;
gridBagConstraints.gridy = 7;
gridBagConstraints.anchor = java.awt.GridBagConstraints.EAST;
gridBagConstraints.insets = new java.awt.Insets(1, 5, 10, 5);
getContentPane().add(maoDeObraLabel, gridBagConstraints);
```

```
maoDeObraTextField.setColumns(8);
maoDeObraTextField.setPreferredSize(new java.awt.Dimension(80, 20));
gridBagConstraints = new java.awt.GridBagConstraints();
gridBagConstraints.gridx = 1;
gridBagConstraints.gridy = 7;
gridBagConstraints.anchor = java.awt.GridBagConstraints.WEST;
gridBagConstraints.insets = new java.awt.Insets(1, 5, 10, 5);
getContentPane().add(maoDeObraTextField, gridBagConstraints);
```

```
pack();
```

```
// </editor-fold>//GEN-END:initComponents
```

```
private void inserirPecas(java.awt.event.ActionEvent evt) {
    Pecas pecas = obterPecasInformada();
    String mensagem_erro = null;

    if (pecas != null)
        mensagem_erro = controlador.inserirPecas(pecas);
    else
        mensagem_erro = "Algum atributo da Peça não foi informado";

    if (mensagem_erro == null) {
        Pecas visao = pecas.getVisao();
```

```

        pecas_cadastradasComboBox.addItem(visao);
        pecas_cadastradasComboBox.setSelectedItem(visao);
    } else {
        informarErro(mensagem_erro);
    }
}

```

```

private void alterarPecas(java.awt.event.ActionEvent evt) {
    Pecas pecas = obterPecasInformada();
    String mensagem_erro = null;

    if (pecas != null)
        mensagem_erro = controlador.alterarPecas(pecas);
    else
        mensagem_erro = "Algum atributo da Peça não foi informado";

    if (mensagem_erro == null) {
        Pecas visao = getVisaoAlterada(pecas.getNome());

        if (visao != null) {
            visao.setCodigo(pecas.getCodigo());
            visao.setCategoria(pecas.getCategoria());
            visao.setPreco(pecas.getPreco());
            visao.setTipo(pecas.getTipo());
            visao.setCor(pecas.getCor());
            visao.setMaoDeObra(pecas.getMaoDeObra());

            pecas_cadastradasComboBox.updateUI();
            pecas_cadastradasComboBox.setSelectedItem(visao);
        }
    } else {
        informarErro(mensagem_erro);
    }
}

```

```

private void consultarPecas(java.awt.event.ActionEvent evt) {
    Pecas visao =

```

```

        (Pecas) pecas_cadastradasComboBox.getSelectedItem());

Pecas pecas = null;
String mensagem_erro = null;

if (visao != null) {
    pecas = Pecas.buscarPecas(visao.getNome());

    if (pecas == null)
        mensagem_erro = "Peça não cadastrada";
} else {
    mensagem_erro = "Nenhuma Peça selecionada";
}

if (mensagem_erro == null) {

    codigoTextField.setText(
        String.valueOf(pecas.getCodigo())
    );

    nomeTextField.setText(pecas.getNome());
    categoriaTextField.setText(pecas.getCategoria());

    precoTextField.setText(
        String.valueOf(pecas.getPreco())
    );

    tipoTextField.setText(pecas.getTipo());
    corTextField.setText(pecas.getCor());

    maoDeObraTextField.setText(
        pecas.getMaoDeObra() ? "S" : "N"
    );

} else {
    informarErro(mensagem_erro);
}

```

```
}
```

```
private void removerPecas(java.awt.event.ActionEvent evt) {  
    Pecas visao =  
        (Pecas) pecas_cadastradasComboBox.getSelectedItem();  
  
    String mensagem_erro = null;  
  
    if (visao != null)  
        mensagem_erro =  
            controlador.removerPecas(visao.getNome());  
    else  
        mensagem_erro = "Nenhuma Peça selecionada";  
  
    if (mensagem_erro == null) {  
        pecas_cadastradasComboBox.removeItem(visao);  
        limparCampos(evt);  
    } else {  
        informarErro(mensagem_erro);  
    }  
}
```

```
private void limparCampos(java.awt.event.ActionEvent evt) {  
    codigoTextField.setText("");  
    nomeTextField.setText("");  
    categoriaTextField.setText("");  
    precoTextField.setText("");  
    tipoTextField.setText("");  
    corTextField.setText("");  
    maoDeObraTextField.setText("");  
}
```

```
// Variables declaration - do not modify//GEN-BEGIN:variables  
private javax.swing.JButton alterarPecas;  
private javax.swing.JPanel comandosPanel;  
private javax.swing.JButton consultarPecas;  
private javax.swing.JLabel codigoLabel;
```

```
private javax.swing.JTextField codigoTextField;
private javax.swing.JLabel nomeLabel;
private javax.swing.JTextField nomeTextField;
private javax.swing.JLabel categoriaLabel;
private javax.swing.JTextField categoriaTextField;
private javax.swing.JLabel precoLabel;
private javax.swing.JTextField precoTextField;
private javax.swing.JLabel tipoLabel;
private javax.swing.JTextField tipoTextField;
private javax.swing.JLabel corLabel;
private javax.swing.JTextField corTextField;
private javax.swing.JLabel maoDeObraLabel;
private javax.swing.JTextField maoDeObraTextField;
private javax.swing.JButton inserirPecas;
private javax.swing.JButton limparCampos;
private javax.swing.JLabel pecasCadastradasLabel;
private javax.swing.JComboBox pecas_cadastradasComboBox;
private javax.swing.JButton removerPecas;
// End of variables declaration//GEN-END:variables
}
```

\$\$\$ src.interfaces.JanelaCadastroSeguradoras

```
package interfaces;

import javax.swing.JOptionPane;
import controles.ControladorCadastroSeguradoras;
import entidades.Seguradora;
import javax.swing.DefaultComboBoxModel;
import java.awt.Frame;

public class JanelaCadastroSeguradoras extends javax.swing.JFrame {

    ControladorCadastroSeguradoras controlador;
    Seguradora[] seguradoras_cadastradas;

    public JanelaCadastroSeguradoras(ControladorCadastroSeguradoras controlador) {
        this(controlador, null);
    }

    public JanelaCadastroSeguradoras(
        ControladorCadastroSeguradoras controlador,
        Frame owner
    ) {
        this.controlador = controlador;
        seguradoras_cadastradas = Seguradora.getVisoes();
        initComponents();
        setSize(new java.awt.Dimension(700, 460));
        configurarJanelaDependente(owner);
        limparCampos(null);
    }

    private void configurarJanelaDependente(Frame owner) {
        setAutoRequestFocus(true);
        setAlwaysOnTop(true);

        if (owner == null) {
            setLocationByPlatform(true);
        }
    }
}
```

```

        return;
    }

    posicionarRelativaAoOwner(owner);
}

private void posicionarRelativaAoOwner(Frame owner) {
    int x = owner.getX() + (owner.getWidth() - getWidth()) / 2;
    int y = owner.getY() + (owner.getHeight() - getHeight()) / 2;
    setLocation(x, y);
}

private void informarErro(String mensagem) {
    JOptionPane.showMessageDialog(
        this,
        mensagem,
        "Erro",
        JOptionPane.ERROR_MESSAGE
    );
}

private Seguradora obterSeguradoraInformada() {

    String nome = nomeTextField.getText();
    if (nome.isEmpty()) return null;

    String cidade = cidadeTextField.getText();
    if (cidade.isEmpty()) cidade = null;

    String cobertura_str = coberturaPercentualTextField.getText();
    if (cobertura_str.isEmpty()) return null;

    double cobertura_percentual;

    try {
        cobertura_percentual =
            Double.parseDouble(cobertura_str.replace(",", "."));
    }

```

```

    } catch (NumberFormatException e) {
        return null;
    }

```

```

return new Seguradora(
    nome,
    cidade,
    cobertura_percentual
);
}

```

```

private Seguradora getVisaoAlterada(String nome) {
    for (Seguradora visao : seguradoras_cadastradas) {
        if (visao.getNome().equals(nome)) return visao;
    }
    return null;
}

```

```

@SuppressWarnings("unchecked")

```

```

// <editor-fold defaultstate="collapsed" desc="Generated Code"> //GEN-BEGIN: initComponents

```

```

private void initComponents() {
    java.awt.GridBagConstraints gridBagConstraints;

    comandosPanel = new javax.swing.JPanel();
    inserirSeguradora = new javax.swing.JButton();
    alterarSeguradora = new javax.swing.JButton();
    consultarSeguradora = new javax.swing.JButton();
    removerSeguradora = new javax.swing.JButton();
    limparCampos = new javax.swing.JButton();
    nomeLabel = new javax.swing.JLabel();
    nomeTextField = new javax.swing.JTextField();
    cidadeLabel = new javax.swing.JLabel();
    cidadeTextField = new javax.swing.JTextField();
    coberturaPercentualLabel = new javax.swing.JLabel();
    coberturaPercentualTextField = new javax.swing.JTextField();
    seguradorasCadastradasLabel = new javax.swing.JLabel();
    seguradoras_cadastradasComboBox = new javax.swing.JComboBox();
}

```

```
setDefaultCloseOperation(javax.swing.WindowConstants.DISPOSE_ON_CLOSE);
setTitle("Cadastrar Seguradoras");
setMinimumSize(new java.awt.Dimension(620, 360));
setPreferredSize(new java.awt.Dimension(700, 460));
getContentPane().setLayout(new java.awt.GridBagLayout());
```

```
inserirSeguradora.setText("Inserir");
inserirSeguradora.addActionListener(new java.awt.event.ActionListener() {
    public void actionPerformed(java.awt.event.ActionEvent evt) {
        inserirSeguradora(evt);
    }
});
comandosPanel.add(inserirSeguradora);
```

```
alterarSeguradora.setText("Alterar");
alterarSeguradora.addActionListener(new java.awt.event.ActionListener() {
    public void actionPerformed(java.awt.event.ActionEvent evt) {
        alterarSeguradora(evt);
    }
});
comandosPanel.add(alterarSeguradora);
```

```
consultarSeguradora.setText("Consultar");
consultarSeguradora.addActionListener(new java.awt.event.ActionListener() {
    public void actionPerformed(java.awt.event.ActionEvent evt) {
        consultarSeguradora(evt);
    }
});
comandosPanel.add(consultarSeguradora);
```

```
removerSeguradora.setText("Remover");
removerSeguradora.addActionListener(new java.awt.event.ActionListener() {
    public void actionPerformed(java.awt.event.ActionEvent evt) {
        removerSeguradora(evt);
    }
});
```

```
comandosPanel.add(removerSeguradora);
```

```
limparCampos.setText("Limpar");
```

```
limparCampos.addActionListener(new java.awt.event.ActionListener() {  
    public void actionPerformed(java.awt.event.ActionEvent evt) {  
        limparCampos(evt);  
    }  
});
```

```
comandosPanel.add(limparCampos);
```

```
gridBagConstraints = new java.awt.GridBagConstraints();  
gridBagConstraints.gridx = 1;  
gridBagConstraints.gridy = 5;  
gridBagConstraints.insets = new java.awt.Insets(5, 5, 5, 5);  
getContentPane().add(comandosPanel, gridBagConstraints);
```

```
nomeLabel.setText("Nome");  
nomeLabel.setMaximumSize(null);  
nomeLabel.setMinimumSize(null);  
gridBagConstraints = new java.awt.GridBagConstraints();  
gridBagConstraints.gridx = 0;  
gridBagConstraints.gridy = 1;  
gridBagConstraints.anchor = java.awt.GridBagConstraints.EAST;  
gridBagConstraints.insets = new java.awt.Insets(1, 16, 10, 10);  
getContentPane().add(nomeLabel, gridBagConstraints);
```

```
nomeTextField.setColumns(24);  
nomeTextField.setPreferredSize(new java.awt.Dimension(240, 20));  
nomeTextField.addActionListener(new java.awt.event.ActionListener() {  
    public void actionPerformed(java.awt.event.ActionEvent evt) {  
        nomeTextFieldActionPerformed(evt);  
    }  
});  
gridBagConstraints = new java.awt.GridBagConstraints();  
gridBagConstraints.gridx = 1;  
gridBagConstraints.gridy = 1;  
gridBagConstraints.ipadx = 80;
```

```
gridBagConstraints.anchor = java.awt.GridBagConstraints.WEST;
gridBagConstraints.insets = new java.awt.Insets(1, 5, 10, 5);
getContentPane().add(nomeTextField, gridBagConstraints);
```

```
cidadeLabel.setText("Cidade");
gridBagConstraints = new java.awt.GridBagConstraints();
gridBagConstraints.gridx = 0;
gridBagConstraints.gridy = 2;
gridBagConstraints.anchor = java.awt.GridBagConstraints.EAST;
gridBagConstraints.insets = new java.awt.Insets(1, 16, 10, 10);
getContentPane().add(cidadeLabel, gridBagConstraints);
```

```
cidadeTextField.setColumns(18);
cidadeTextField.setPreferredSize(new java.awt.Dimension(180, 20));
cidadeTextField.addActionListener(new java.awt.event.ActionListener() {
    public void actionPerformed(java.awt.event.ActionEvent evt) {
        cidadeTextFieldActionPerformed(evt);
    }
});
gridBagConstraints = new java.awt.GridBagConstraints();
gridBagConstraints.gridx = 1;
gridBagConstraints.gridy = 2;
gridBagConstraints.ipadx = 20;
gridBagConstraints.anchor = java.awt.GridBagConstraints.WEST;
gridBagConstraints.insets = new java.awt.Insets(1, 5, 10, 5);
getContentPane().add(cidadeTextField, gridBagConstraints);
```

```
coberturaPercentualLabel.setText("Cobertura Percentual");
gridBagConstraints = new java.awt.GridBagConstraints();
gridBagConstraints.gridx = 0;
gridBagConstraints.gridy = 3;
gridBagConstraints.anchor = java.awt.GridBagConstraints.EAST;
gridBagConstraints.insets = new java.awt.Insets(1, 16, 10, 10);
getContentPane().add(coberturaPercentualLabel, gridBagConstraints);
```

```
coberturaPercentualTextField.setColumns(10);
coberturaPercentualTextField.setPreferredSize(new java.awt.Dimension(100, 20));
```

```

coberturaPercentualTextField.addActionListener(new java.awt.event.ActionListener() {
    public void actionPerformed(java.awt.event.ActionEvent evt) {
        coberturaPercentualTextFieldActionPerformed(evt);
    }
});

```

```

gridBagConstraints = new java.awt.GridBagConstraints();
gridBagConstraints.gridx = 1;
gridBagConstraints.gridy = 3;
gridBagConstraints.ipadx = 0;
gridBagConstraints.anchor = java.awt.GridBagConstraints.WEST;
gridBagConstraints.insets = new java.awt.Insets(1, 5, 10, 5);
getContentPane().add(coberturaPercentualTextField, gridBagConstraints);

```

```

seguradorasCadastradasLabel.setText("Seguradoras Cadastradas");
gridBagConstraints = new java.awt.GridBagConstraints();
gridBagConstraints.anchor = java.awt.GridBagConstraints.EAST;
gridBagConstraints.insets = new java.awt.Insets(0, 12, 0, 5);
getContentPane().add(seguradorasCadastradasLabel, gridBagConstraints);

```

```

seguradoras_cadastradasComboBox.setModel(new
DefaultComboBoxModel(seguradoras_cadastradas));
seguradoras_cadastradasComboBox.addActionListener(new java.awt.event.ActionListener() {
    public void actionPerformed(java.awt.event.ActionEvent evt) {
        seguradoras_cadastradasComboBoxActionPerformed(evt);
    }
});

```

```

gridBagConstraints = new java.awt.GridBagConstraints();
gridBagConstraints.gridx = 1;
gridBagConstraints.gridy = 0;
gridBagConstraints.anchor = java.awt.GridBagConstraints.WEST;
gridBagConstraints.insets = new java.awt.Insets(1, 5, 10, 5);
getContentPane().add(seguradoras_cadastradasComboBox, gridBagConstraints);

```

```

pack();

```

```

} // </editor-fold> // GEN-END: initComponents

```

```

private void nomeTextFieldActionPerformed(java.awt.event.ActionEvent evt) { // GEN-

```

```

FIRST:event_nomeTextFieldActionPerformed
    // TODO add your handling code here:
} //GEN-LAST:event_nomeTextFieldActionPerformed

private void cidadeTextFieldActionPerformed(java.awt.event.ActionEvent evt) { //GEN-FIRST:event_cidadeTextFieldActionPerformed
    // TODO add your handling code here:
} //GEN-LAST:event_cidadeTextFieldActionPerformed

private void coberturaPercentualTextFieldActionPerformed(java.awt.event.ActionEvent evt) { //GEN-FIRST:event_coberturaPercentualTextFieldActionPerformed
    // TODO add your handling code here:
} //GEN-LAST:event_coberturaPercentualTextFieldActionPerformed

private void alterarSeguradora(java.awt.event.ActionEvent evt) { //GEN-FIRST:event_alterarSeguradora
    Seguradora seguradora =
        obterSeguradoraInformada();

    String mensagem_erro = null;

    if (seguradora != null)
        mensagem_erro =
            controlador.alterarSeguradora(seguradora);
    else
        mensagem_erro =
            "Algum atributo da Seguradora não foi informado.";

    if (mensagem_erro == null) {

        Seguradora visao =
            getVisaoAlterada(seguradora.getNome());

        if (visao != null) {

            visao.setCidade(
                seguradora.getCidade()
            );

```

```

        visao.setCoberturaPercentual(
            seguradora.getCoberturaPercentual()
        );

        seguradoras_cadastradasComboBox.updateUI();

        seguradoras_cadastradasComboBox
            .setSelectedItem(visao);
    }

} else {
    informarErro(mensagem_erro);
}
} //GEN-LAST:event_alterarSeguradora

```

```

private void inserirSeguradora(java.awt.event.ActionEvent evt) { //GEN-FIRST:event_inserirSeguradora
    Seguradora seguradora =
        obterSeguradoraInformada();

    String mensagem_erro = null;

    if (seguradora != null)
        mensagem_erro =
            controlador.inserirSeguradora(seguradora);
    else
        mensagem_erro =
            "Algum atributo da Seguradora não foi informado";

    if (mensagem_erro == null) {

        Seguradora visao =
            seguradora.getVisao();

        seguradoras_cadastradasComboBox
            .addItem(visao);
    }
} //GEN-LAST:event_inserirSeguradora

```

```

        seguradoras_cadastradasComboBox
            .setSelectedItem(visao);

    } else {
        informarErro(mensagem_erro);
    }
} //GEN-LAST:event_inserirSeguradora

private void consultarSeguradora(java.awt.event.ActionEvent evt) { //GEN-FIRST:event_consultarSeguradora
    Seguradora visao =
        (Seguradora)
        seguradoras_cadastradasComboBox
            .getSelectedItem();

    Seguradora seguradora = null;

    String mensagem_erro = null;

    if (visao != null) {

        seguradora =
            Seguradora.buscarSeguradora(
                visao.getNome()
            );

        if (seguradora == null)
            mensagem_erro =
                "Seguradora não cadastrada";

    } else {
        mensagem_erro =
            "Nenhuma Seguradora selecionada";
    }

    if (mensagem_erro == null) {

```

```
nomeTextField.setText(  
    seguradora.getNome()  
);
```

```
String cidade =  
    seguradora.getCidade();
```

```
if (cidade == null)  
    cidade = "";
```

```
cidadeTextField.setText(cidade);
```

```
coberturaPercentualTextField.setText(  
    String.valueOf(  
        seguradora  
            .getCoberturaPercentual()  
    )  
);
```

```
} else {  
    informarErro(mensagem_erro);  
}  
} //GEN-LAST:event_consultarSeguradora
```

```
private void removerSeguradora(java.awt.event.ActionEvent evt) { //GEN-FIRST:event_removeSeguradora
```

```
    Seguradora visao =  
        (Seguradora)  
        seguradoras_cadastradasComboBox  
            .getSelectedItem();
```

```
String mensagem_erro = null;
```

```
if (visao != null)  
    mensagem_erro =  
        controlador.removerSeguradora(
```

```

        visao.getNome()
    );
else
    mensagem_erro =
        "Nenhuma Seguradora selecionada";

if (mensagem_erro == null) {

    seguradoras_cadastradasComboBox
        .removeItem(visao);

    limparCampos(evt);

} else {
    informarErro(mensagem_erro);
}
} //GEN-LAST:event_removeSeguradora

private void limparCampos(java.awt.event.ActionEvent evt) { //GEN-
FIRST:event_limparCampos
    nomeTextField.setText("");
    cidadeTextField.setText("");
    coberturaPercentualTextField.setText("");
} //GEN-LAST:event_limparCampos

private void seguradoras_cadastradasComboBoxActionPerformed(java.awt.event.ActionEvent
evt) { //GEN-FIRST:event_seguradoras_cadastradasComboBoxActionPerformed

} //GEN-LAST:event_seguradoras_cadastradasComboBoxActionPerformed

// Variables declaration - do not modify //GEN-BEGIN:variables
private javax.swing.JButton alterarSeguradora;
private javax.swing.JLabel cidadeLabel;
private javax.swing.JTextField cidadeTextField;
private javax.swing.JLabel coberturaPercentualLabel;
private javax.swing.JTextField coberturaPercentualTextField;

```

```
private javax.swing.JPanel comandosPanel;
private javax.swing.JButton consultarSeguradora;
private javax.swing.JButton inserirSeguradora;
private javax.swing.JButton limparCampos;
private javax.swing.JLabel nomeLabel;
private javax.swing.JTextField nomeTextField;
private javax.swing.JButton removerSeguradora;
private javax.swing.JLabel seguradorasCadastradasLabel;
private javax.swing.JComboBox seguradoras_cadastradasComboBox;
// End of variables declaration//GEN-END:variables
}
```

\$\$\$ src.interfaces.JanelaSistema

```
package interfaces;
```

```
import controles.ControladorCadastroSeguradoras;
```

```
import controles.ControladorCadastroPecas;
```

```
import javax.swing.JOptionPane;
```

```
import persistência.BD;
```

```
public class JanelaSistema extends javax.swing.JFrame {
```

```
    public JanelaSistema() {
```

```
        BD.criaConexao();
```

```
        initComponents();
```

```
    }
```

```
@SuppressWarnings("unchecked")
```

```
private void informarServiçoIndisponível() {
```

```
    JOptionPane.showMessageDialog(
```

```
        this,
```

```
        "Serviço Indisponível",
```

```
        "Informação",
```

```
        JOptionPane.INFORMATION_MESSAGE
```

```
    );
```

```
}
```

```
// <editor-fold defaultstate="collapsed" desc="Generated Code">//GEN-BEGIN: initComponents
```

```
private void initComponents() {
```

```
    seguradoras_orçamentoMenuBar = new javax.swing.JMenuBar();
```

```
    sinistroMenu = new javax.swing.JMenu();
```

```
    cadastrar_sinistroMenuItem = new javax.swing.JMenuItem();
```

```
    seguradoraMenu = new javax.swing.JMenu();
```

```
    cadastrar_seguradoraItemMenu = new javax.swing.JMenuItem();
```

```
    peçaMenu = new javax.swing.JMenu();
```

```
    cadastrar_peçaMenuItem = new javax.swing.JMenuItem();
```

```
orçamentoMenu = new javax.swing.JMenu();
cadastrar_orçamentoMenuItem = new javax.swing.JMenuItem();
pesquisar_orçamentoMenuItem = new javax.swing.JMenuItem();

setDefaultCloseOperation(javax.swing.WindowConstants.DISPOSE_ON_CLOSE);
setTitle("Cadastro de Seguradoras e Peças");
setAlwaysOnTop(true);
addWindowListener(new java.awt.event.WindowAdapter() {
    public void windowClosed(java.awt.event.WindowEvent evt) {
        terminarSistema(evt);
    }
});

sinistroMenu.setText("Sinistro");

cadastrar_sinistroMenuItem.setText("Cadastrar");
cadastrar_sinistroMenuItem.addActionListener(new java.awt.event.ActionListener() {
    public void actionPerformed(java.awt.event.ActionEvent evt) {
        cadastrarSinistro(evt);
    }
});
sinistroMenu.add(cadastrar_sinistroMenuItem);

seguradoras_orçamentoMenuBar.add(sinistroMenu);

seguradoraMenu.setText("Seguradora");

cadastrar_seguradoraItemMenu.setText("Cadastrar");
cadastrar_seguradoraItemMenu.addActionListener(new java.awt.event.ActionListener() {
    public void actionPerformed(java.awt.event.ActionEvent evt) {
        cadastrarSeguradora(evt);
    }
});
seguradoraMenu.add(cadastrar_seguradoraItemMenu);

seguradoras_orçamentoMenuBar.add(seguradoraMenu);
```

```
peçaMenu.setText("Peça");
```

```
cadastrar_peçaMenuItem.setText("Cadastrar");
```

```
cadastrar_peçaMenuItem.addActionListener(new java.awt.event.ActionListener() {  
    public void actionPerformed(java.awt.event.ActionEvent evt) {  
        cadastrarPeça(evt);  
    }  
});  
peçaMenu.add(cadastrar_peçaMenuItem);
```

```
seguradoras_orçamentoMenuBar.add(peçaMenu);
```

```
orçamentoMenu.setText("Orçamento");
```

```
cadastrar_orçamentoMenuItem.setText("Cadastrar");
```

```
cadastrar_orçamentoMenuItem.addActionListener(new java.awt.event.ActionListener() {  
    public void actionPerformed(java.awt.event.ActionEvent evt) {  
        cadastrarOrçamentos(evt);  
    }  
});  
orçamentoMenu.add(cadastrar_orçamentoMenuItem);
```

```
pesquisar_orçamentoMenuItem.setText("Pesquisar");
```

```
pesquisar_orçamentoMenuItem.addActionListener(new java.awt.event.ActionListener() {  
    public void actionPerformed(java.awt.event.ActionEvent evt) {  
        pesquisarOrçamentos(evt);  
    }  
});  
orçamentoMenu.add(pesquisar_orçamentoMenuItem);
```

```
seguradoras_orçamentoMenuBar.add(orçamentoMenu);
```

```
setJMenuBar(seguradoras_orçamentoMenuBar);
```

```
javax.swing.GroupLayout layout = new javax.swing.GroupLayout(getContentPane());  
getContentPane().setLayout(layout);  
layout.setHorizontalGroup(
```

```

        layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
            .addGap(0, 400, Short.MAX_VALUE)
    );
    layout.setVerticalGroup(
        layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
            .addGap(0, 278, Short.MAX_VALUE)
    );

    pack();
} // </editor-fold> // GEN-END: initComponents

private void cadastrarSeguradora(java.awt.event.ActionEvent evt) { // GEN-
FIRST:event_cadastrarSeguradora
    new ControladorCadastroSeguradoras(this);
} // GEN-LAST:event_cadastrarSeguradora

private void terminarSistema(java.awt.event.WindowEvent evt) { // GEN-
FIRST:event_terminarSistema
    BD.fechaConexao();
    System.exit(0);
} // GEN-LAST:event_terminarSistema

private void pesquisarOrçamentos(java.awt.event.ActionEvent evt) { // GEN-
FIRST:event_pesquisarOrçamentos
    informarServiçoIndisponível();
} // GEN-LAST:event_pesquisarOrçamentos

private void cadastrarPeça(java.awt.event.ActionEvent evt) { // GEN-FIRST:event_cadastrarPeça
    new ControladorCadastroPecas(this);
} // GEN-LAST:event_cadastrarPeça

private void cadastrarOrçamentos(java.awt.event.ActionEvent evt) { // GEN-
FIRST:event_cadastrarOrçamentos
    informarServiçoIndisponível();
} // GEN-LAST:event_cadastrarOrçamentos

private void cadastrarSinistro(java.awt.event.ActionEvent evt) { // GEN-
FIRST:event_cadastrarSinistro

```

```
    informarServiçoIndisponível();  
} //GEN-LAST:event_cadastrarSinistro
```

```
public static void main(String args[]) {  
    java.awt.EventQueue.invokeLater(new Runnable() {  
        public void run() {  
            new JanelaSistema().setVisible(true);  
        }  
    });  
}
```

```
// Variables declaration - do not modify//GEN-BEGIN:variables  
private javax.swing.JMenuItem cadastrar_orçamentoMenuItem;  
private javax.swing.JMenuItem cadastrar_peçaMenuItem;  
private javax.swing.JMenuItem cadastrar_seguradoraItemMenu;  
private javax.swing.JMenuItem cadastrar_sinistroMenuItem;  
private javax.swing.JMenu orçamentoMenu;  
private javax.swing.JMenuItem pesquisar_orçamentoMenuItem;  
private javax.swing.JMenu peçaMenu;  
private javax.swing.JMenu seguradoraMenu;  
private javax.swing.JMenuBar seguradoras_orçamentoMenuBar;  
private javax.swing.JMenu sinistroMenu;  
// End of variables declaration//GEN-END:variables
```

```
}
```

\$\$\$ src.persistência.BD

```
package persistência;

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;

public class BD {

    static final String URL_BD = "jdbc:mysql://localhost/banco";
    static final String USUARIO = "root";
    static final String SENHA = "admin";
    public static Connection conexao = null;

    public static void criaConexao() {
        try {
            conexao = DriverManager.getConnection(URL_BD, USUARIO, SENHA);
        } catch (SQLException excecao_sql) {
            excecao_sql.printStackTrace();
        }
    }

    public static void fechaConexao() {
        try {
            if (conexao != null) {
                conexao.close();
            }
        } catch (SQLException excecao_sql) {
            excecao_sql.printStackTrace();
        }
    }
}
```